
Causal RCA Toolbox

Causality Analysis & Root-Cause Fault Identification

Operator and Engineer Manual



Companion tool to the FDI Analytics Toolbox

Version 2.0

August 6, 2026

Contents

Introduction	3
1 Data & Scenario	7
1.1 Loading data	7
1.2 Baseline and fault windows	7
1.3 Variable selection	8
1.4 Hand-off from the FDI toolbox	9
1.5 Common problems	9
2 Pre-processing	10
2.1 Why conditioning the signals matters here	10
2.2 The six steps	11
2.3 Stationarity: reading ADF and KPSS	11
2.3.1 A worked example: why a step fault fails the test	12
2.4 Recommended defaults	13
3 Causal Discovery	14
3.1 The single idea behind almost every method	14
3.2 Direct influence versus indirect influence	14
3.3 Domain knowledge	15
3.3.1 The three constraint types	15
3.3.2 How constraints flow through the toolbox	16
3.4 Method families	17
3.4.1 Classic / statistical	17
3.4.2 Machine learning — traditional	18
3.4.3 Machine learning — deep learning	19
3.4.4 Probabilistic	19
3.4.5 Choosing between the families	20
3.5 Parameters	21
3.6 Reading the causal graph	21
3.6.1 Full path listing	23
3.7 If the graph looks wrong	23
4 Monitoring	24
4.1 What a structural shift is — and is not	24
4.1.1 Why the change is spread over several windows	24
4.2 Settings	25
4.3 Reading the two plots	26
4.4 Link activity	26
4.5 If something looks wrong	26
5 Root-Cause Identification	30
5.1 How candidates are ranked	30
5.2 The biggest deviation is usually not the cause	31
5.3 Propagation path	32
5.3.1 Reading the chains operationally	32
5.4 Plain-language summary	32
5.4.1 What it contains	33

5.4.2	Why it is generated deterministically	33
5.4.3	Optional AI rewrite	34
5.5	Understanding the score in the interface	34
6	Evaluation & Report	35
6.1	The central difficulty	35
6.2	Reliability — validation without ground truth	35
6.2.1	Out-of-sample R^2	35
6.2.2	Predictive gain	36
6.2.3	Bootstrap stability	36
6.2.4	Placebo test	36
6.2.5	Reading the four together	36
6.3	Structural accuracy — reference required	37
6.3.1	A worked comparison	38
6.4	The report	38
7	The AI Assistant	40
7.1	Configuration	40
7.2	What the assistant knows	41
7.3	When to use it	41
7.4	When <i>not</i> to rely on it	41
8	Running and Deploying the Toolbox	43
8.1	Development mode versus production	43
8.2	The developer panel	43
8.3	What to expect in terms of run time	44
8.4	Data handling	44
9	Reference	45
9.1	Choosing a method	45
9.2	Recommended workflow	45
9.3	Troubleshooting	45
9.4	Interpretation and limitations	46
9.5	Glossary	47
9.6	Software requirements	47

Introduction

What this tool does

Industrial plants generate hundreds of correlated sensor signals. When something goes wrong, many of them deviate at once — but only one is usually the origin. The others are carrying the same disturbance downstream, and they often carry it *more visibly* than the sensor that failed.

This is why correlation is not enough. Two signals that move together tell you nothing about which one moved first, which one drove the other, or whether both are being driven by something you did not measure. Separating a cause from its consequences requires three things that correlation does not provide:

- **Direction** — did A drive B , or B drive A ?
- **Time** — causes precede effects, and the delay between them is itself diagnostic information.
- **Conditioning** — if $A \rightarrow B \rightarrow C$, then A and C correlate strongly, yet A does not act on C directly. Only by accounting for B does the indirect link disappear.

The Causal RCA Toolbox takes multivariate time-series data from a process, infers a *directed, time-lagged causal graph* among the variables, and then uses that graph to identify which variable most likely *originated* a fault and how the disturbance *propagated* to everything else.

Where this fits

The FDI Analytics Toolbox answers “*is there a fault, and which sensors are affected?*” This tool answers the next question: “*which sensor caused it, and how did it spread?*” Stage 1 can import the flagged variables and fault window directly from an FDI export, so the two tools chain together.

Who it is for

The tool is designed for two audiences at once:

- **Plant and maintenance operators**, who need a trustworthy answer in plain language. Every method, parameter and metric in the interface carries a plain-language description, and the software writes a narrative summary of its own findings that can go straight into a shift log.
- **Process and control engineers**, who need methodological depth: 19 discovery algorithms across four families, tunable parameters, uncertainty quantification, and validation that works without ground truth.

Both audiences need the same thing from this manual — not just *which button to press*, but *why the answer can be believed*. Every chapter therefore explains the reasoning behind a stage before describing its controls.

How to read this manual

Chapters 1–6 follow the six stages of the pipeline in order. Chapter 7 covers the optional AI assistant, Chapter 8 covers deployment, and Chapter 9 is reference material. Three kinds of box appear throughout:

Explanation

Background that helps a result make sense.

Practice

A recommendation for how to work.

Caution

Something that will produce a wrong or misleading answer if ignored.

Screenshots show the interface as you will see it. Analysis figures are computed from the built-in demonstration dataset described below, so every number in them can be reproduced.

The six stages



Figure 1: The analysis pipeline. Stages 1–3 build the causal graph; Stage 4 monitors it over time; Stage 5 identifies the root cause; Stage 6 validates and exports.

1. **Data & Scenario** — load data, define baseline and fault windows.
2. **Pre-processing** — clean, detrend, filter and scale the signals.
3. **Causal Discovery** — infer the causal graph, with optional domain knowledge.
4. **Monitoring** — track how the causal structure changes over time.
5. **Root-Cause** — rank candidates and trace the propagation path.
6. **Evaluation & Report** — test reliability and export the findings.

Stages must be run in order, because each reads what the previous one stored. Changing something in an early stage does not automatically refresh the later ones — re-run forward from the point you changed.

Getting started

```
pip install -r requirements.txt
python app.py
# open http://127.0.0.1:8051
```

For an installation that operators will use, see Chapter 8; the command above starts a *development* server.

The demonstration dataset

On Stage 1, **Load synthetic demo** generates a process whose true causal structure is known. It is worth understanding, because it is the only setting in which the tool's answers can be checked against the truth — and every analysis figure in this manual comes from it.

Five tags are simulated for 1500 samples. Each variable depends on its own previous value and on the previous value of its parents:

$$\begin{aligned} T1_t &= 0.55 T1_{t-1} + f_t + \varepsilon_t \\ F1_t &= 0.50 F1_{t-1} + 0.80 T1_{t-1} + \varepsilon_t \\ L1_t &= 0.45 L1_{t-1} + 0.75 F1_{t-1} + \varepsilon_t \\ P1_t &= 0.40 P1_{t-1} + 0.65 L1_{t-1} + 0.35 T1_{t-1} + \varepsilon_t \\ A1_t &= 0.40 A1_{t-1} + 0.70 P1_{t-1} + \varepsilon_t \end{aligned}$$

The true structure is therefore a chain with one shortcut, and every influence acts at a delay of exactly one sample:

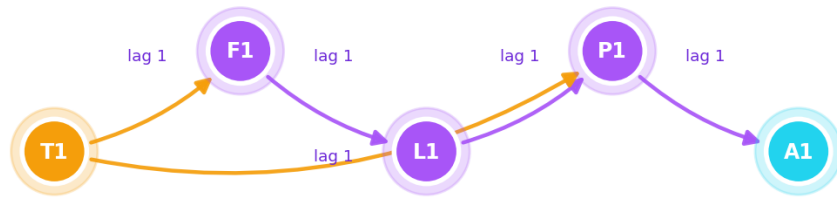


Figure 2: The known structure of the demonstration process: $T1 \rightarrow F1 \rightarrow L1 \rightarrow P1 \rightarrow A1$ with a shortcut $T1 \rightarrow P1$. Colour shows structural role — $T1$ is a driver, $A1$ a pure effect.

The fault f_t is a step of $+1.6$ entering the *root* $T1$ at sample 900 and persisting to the end of the record. Because $T1$ feeds everything else, the step propagates through the chain, amplifying as it goes.

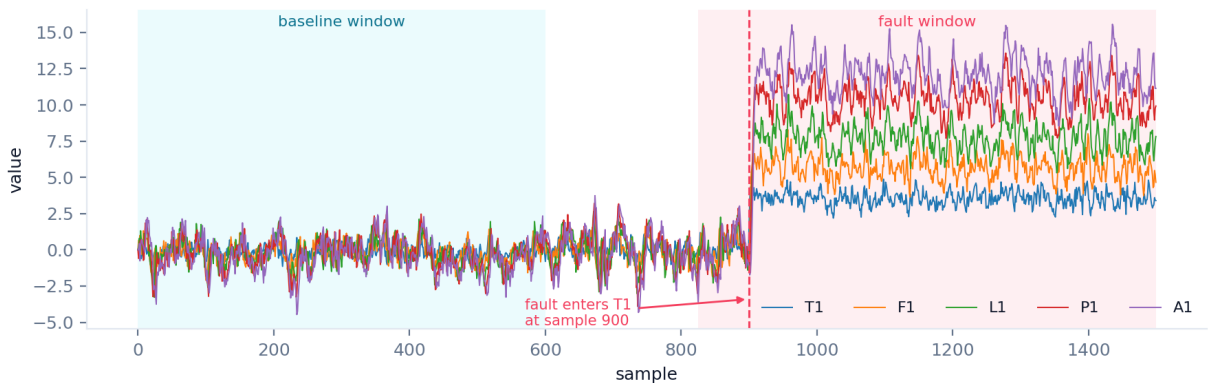


Figure 3: The demonstration dataset with the default analysis windows. The fault enters $T1$ at sample 900, but the sensors furthest downstream shift the most — the central difficulty this tool exists to resolve.

Two properties of this dataset to keep in mind

1. **The fault is additive, not structural.** It changes the *level* of **T1**; it does not change any coefficient. The causal structure after sample 900 is identical to the structure before it. This matters in Stage 4 (Section 4.1).
2. **The true origin deviates least.** As Figure 3 shows, deviation magnitude grows downstream. Any method that ranks candidates by “which sensor moved most” gets this dataset exactly backwards.

STAGE 1

Data & Scenario

The task of this stage is to get data into the tool and to mark which part of the record is normal and which contains the disturbance. Everything downstream is a comparison between those two periods.

STAGE 1

Data & Scenario Setup

Load process time-series, then mark a normal baseline window and a fault window. Root-cause analysis is fundamentally a comparison between these two.

Load data

CSV or Excel - one column per sensor

Drop a file or click to browse
first column may be time/index

— of —

Load synthetic demo (known ground truth)

FDI HAND-OFF

Import flagged variables / fault window from the FDI toolbox (JSON, optional)

Scenario windows

index ranges into the loaded series

No data loaded yet.

Baseline start: 0 Baseline end: 0

Fault start: 0 Fault end: 0

VARIABLES TO ANALYSE

auto: all numeric

Select all Clear

Save scenario →

Load data first.

+ Ask AI

Figure 1.1: Stage 1 before any data is loaded. The left card brings data in; the right card defines the scenario.

1.1 Loading data

The tool accepts **CSV** and **Excel** files in which each column is a sensor tag and each row is a time step. Requirements are deliberately minimal:

- One row per time step, in chronological order, at a constant sampling interval. The tool does not resample irregular timestamps for you — if your historian exports on change-of-value, resample before loading.
- A leading column named **time**, **index** or **t** is treated as the time axis and is *not* offered as a variable. If no such column exists, the row index is used.
- Only columns read as numeric become available for analysis. A column containing units, thousands separators, or a status string will be silently ignored — if a tag is missing from the variable list, this is almost always why.

Alternatively, **Load synthetic demo** generates the validated demonstration process of Section . Because its true causal structure is known, it is the only setting in which the structural accuracy metrics (Section 6.3) are meaningful.

1.2 Baseline and fault windows

Root-cause analysis is fundamentally a *comparison*: how does each sensor behave during the fault relative to how it behaves normally? You therefore define two windows, given as row indices into

the loaded series:

- **Baseline window** — a period of normal operation.
- **Fault window** — the period containing the disturbance.

When data loads, both windows are filled in automatically: the baseline covers the first 40 % of the record, and the fault window runs from 55 % to the last sample. **These defaults are derived from the record length alone.** No detection is involved, and they will only be right by coincidence. Always check them against the preview strip before saving.



Figure 1.2: After loading the demonstration data. The preview shows the baseline window shaded cyan and the fault window red; the confirmation line beneath **Save scenario** states exactly what was stored.

Choosing windows

- Make the baseline long enough to capture normal variability — a few hundred samples at minimum — and representative of the operating point you care about. A baseline that spans a grade change or a shutdown will make normal operation look like a fault.
- Start the fault window *at, or slightly before*, the first visible deviation. Starting late truncates the earliest evidence, which is exactly what the root-cause ranking depends on.
- The windows must not overlap. The application does not enforce this; keep **Fault start** greater than **Baseline end**.

1.3 Variable selection

Select the sensors to include. Fewer, well-chosen variables generally produce a clearer and more reliable graph than including every available tag, for two reasons that pull in the same direction:

- every additional variable adds candidate links, and with them the opportunity for false positives;
- the cost of most methods grows faster than linearly in the number of variables, and conditional methods lose statistical power as the number of conditioning variables rises.

There is a countervailing risk, however, and it is the more serious one: a driver you leave out cannot be discovered, and its influence will be attributed to whichever included variable

correlates with it. **Prefer to omit redundant tags rather than potentially causal ones.** Two transmitters on the same line are redundant; an ambient temperature you think is irrelevant may not be.

1.4 Hand-off from the FDI toolbox

The **FDI hand-off** drop zone accepts the JSON export from the FDI Analytics Toolbox and populates the flagged variables and the detected fault window automatically. This is the intended workflow: detection and isolation happen there, causal attribution happens here, and the fault window does not have to be identified twice.

1.5 Common problems

Symptom	Cause and remedy
<i>“Load data first.”</i> in red	Save scenario was pressed before any data was loaded, or the load failed.
The variable list stays empty	No column was read as numeric. Check the delimiter, the header row, and whether numbers are stored as text.
The preview shows fewer traces than expected	The preview draws at most the first six variables by design. The full selection is still analysed.
The fault band sits on flat signal	The automatic window indices do not suit this record. Read the sample number of the first deviation from the preview axis and enter it manually.
Later stages use old settings	The scenario was edited but not saved. Press Save scenario again, then re-run from Stage 2.

STAGE 2

Pre-processing

2.1 Why conditioning the signals matters here

Every discovery method in Stage 3 asks a version of the same question: *does the past of A help predict the future of B?* That question is only meaningful if two conditions hold.

The series must be stationary. A series is stationary when its statistical properties — mean, variance, autocorrelation — do not change over the record. If two tags both drift upward, they will appear to predict each other regardless of any physical connection, because both are predictable from the trend alone. Trends manufacture causal links that do not exist.

The series must be comparably scaled. Several methods penalise or compare coefficients across variables. If one tag is measured in thousands and another in fractions, the first will dominate on magnitude alone, irrespective of its causal role.

Stage 2 provides a six-step pipeline addressing these and related problems. Each step is optional, each carries an inline plain-language description, and they are always applied **in the order 1 → 6**. That order matters: it makes no sense to detrend before filling gaps, or to scale before removing outliers that would distort the scale.

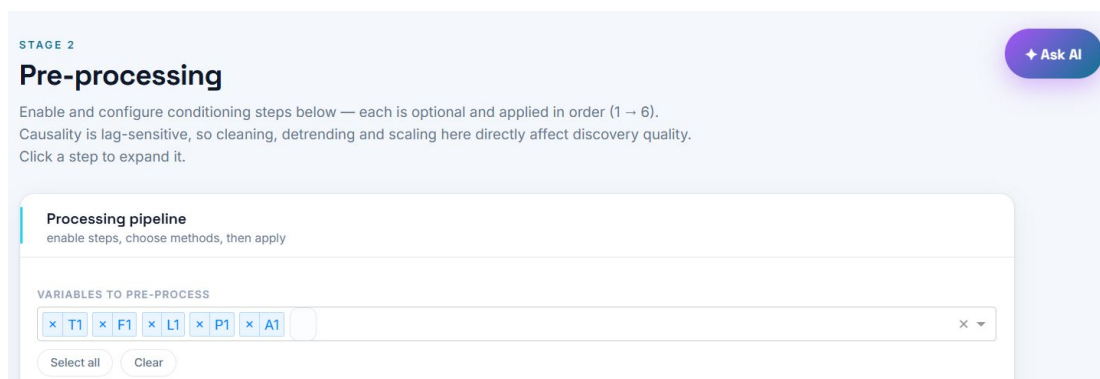


Figure 2.1: Stage 2 as it opens. The variable list is inherited from Stage 1 and can be narrowed further.

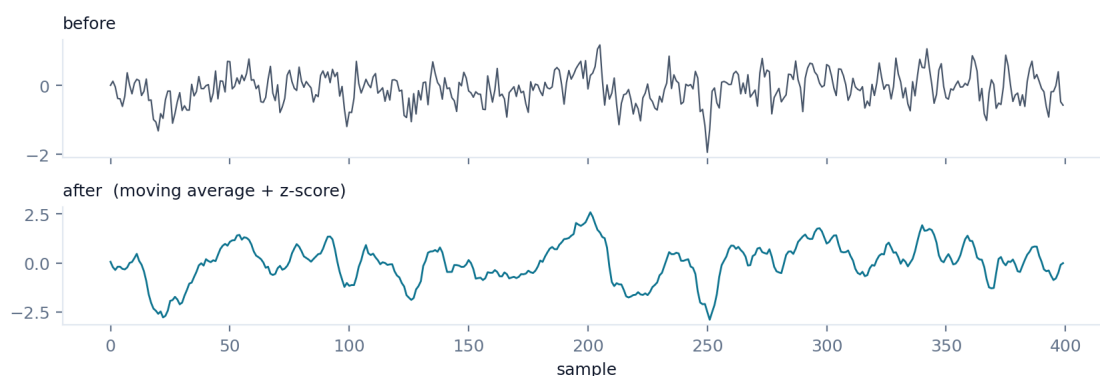


Figure 2.2: Effect of pre-processing on one sensor: a moving-average filter removes measurement noise and z-score scaling standardises the range. The shape and timing of the underlying dynamics are preserved — which is the property to check after any conditioning step.

2.2 The six steps

Step	Methods	Purpose and risk
1 · Missing values	Linear interpolation, forward/back fill, mean imputation, drop rows	Fill gaps so the time axis stays continuous. Dropping rows silently changes the meaning of a lag, since the samples either side of a gap are no longer one interval apart.
2 · Resampling	Keep every k -th sample	Reduce the data rate when sampling is much faster than the process dynamics. Multiplies the physical meaning of one lag by k — revisit Max lag afterwards.
3 · Outliers	Z-score clipping, IQR clipping	Trim spikes and instrument glitches that would otherwise dominate a least-squares fit. Clip too aggressively and genuine process excursions — often the most informative part of the record — are removed with them.
4 · Detrending	Linear detrend, first-order differencing, remove moving-average baseline	Remove drift and level shifts; the key step for stationarity . Differencing also removes the level information, so afterwards the analysis is about rates of change rather than absolute values.
5 · Filtering	Moving average, median filter, exponential smoothing	Suppress measurement noise while preserving dynamics. A long window spreads each sample into its neighbours and manufactures lagged correlation that no physical coupling produced.
6 · Normalisation	Standardisation (z-score), min-max, robust scaling	Put all sensors on a comparable scale. Robust scaling is preferable when outliers remain.

2.3 Stationarity: reading ADF and KPSS

After **Apply pre-processing**, the page reports two tests per variable. They are complementary, and the most common mistake is to read them as if they were the same test.

Test	Null hypothesis	Therefore
ADF	The series has a unit root (<i>non-stationary</i>).	A small p argues <i>for</i> stationarity.
KPSS	The series is stationary.	A small p argues <i>against</i> stationarity.

The two tests together give four possible outcomes, and the verdict column combines them:

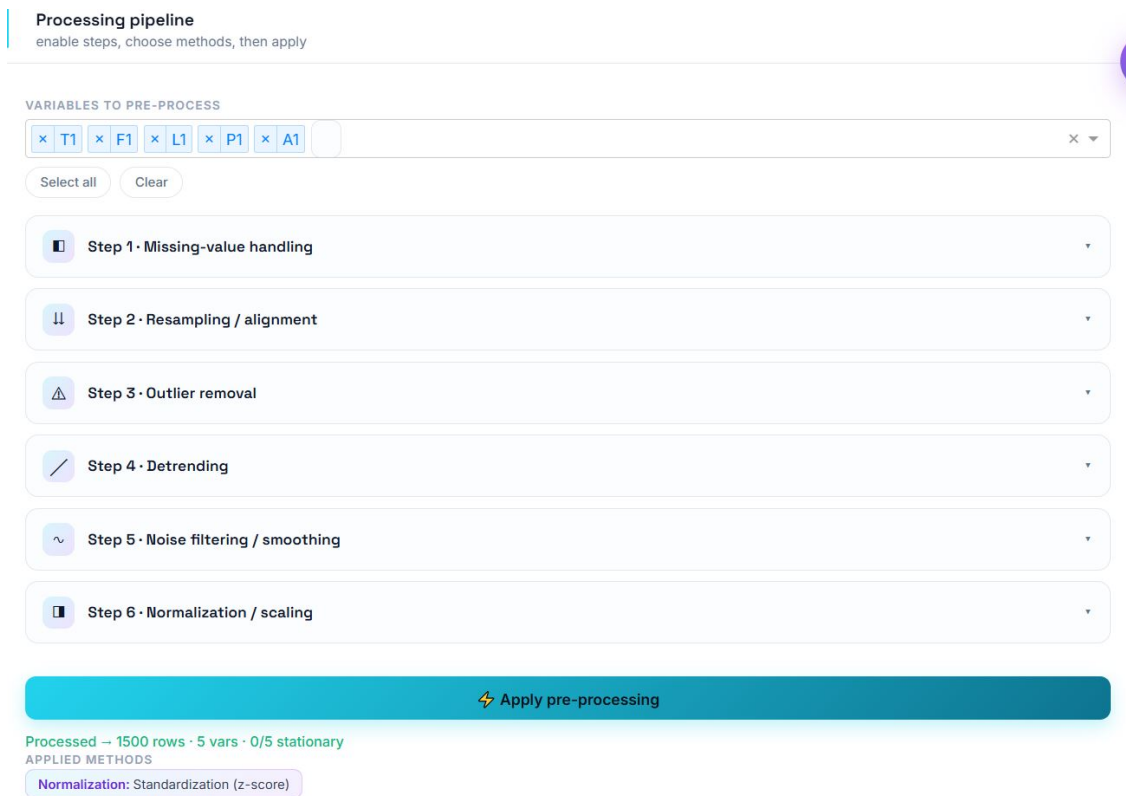


Figure 2.3: The pipeline collapsed. Clicking a row expands it, enables the step and reveals its methods. A step never opened does not run.

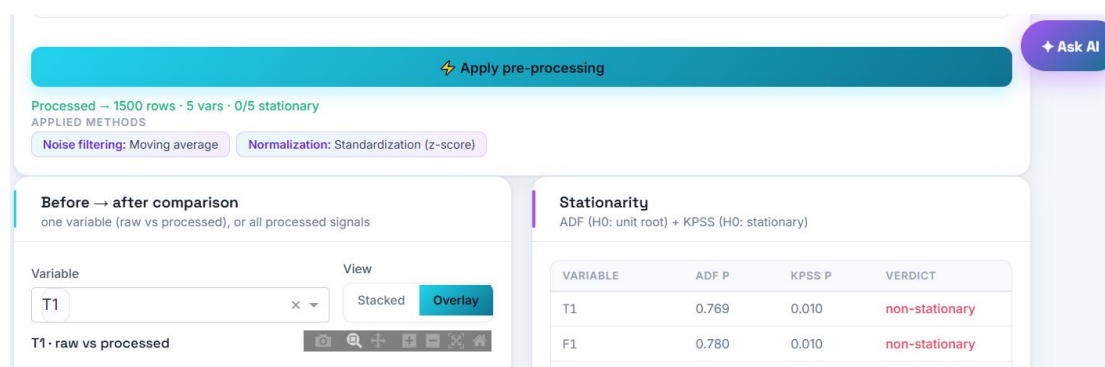
ADF	KPSS	Interpretation
p small	p large	Stationary. Proceed.
p large	p small	Non-stationary. Difference or detrend, then re-check.
p small	p small	Conflicting; often a series that is stationary around a deterministic trend. Try linear detrending rather than differencing.
p large	p large	Inconclusive — usually too few samples, or a very noisy signal. Lengthen the record.

Two practical notes on the numbers

KPSS p -values are clipped by the test's lookup table: a reported 0.010 means “0.01 or less”, not a precise value. And a verdict of n/a means the test could not run — `statsmodels` is not installed, the series is shorter than about 20 samples, or it has zero variance. A constant tag cannot be tested and should be removed from the analysis.

2.3.1 A worked example: why a step fault fails the test

Figure 2.3.1 shows the demonstration data after applying smoothing and z-score scaling only. Every variable is reported non-stationary, with ADF around 0.78 and KPSS at its floor.



Enabling **Step 4** with first-order differencing removes it, and the verdicts flip to stationary. Note also that the tests run over the *whole* record, both windows included, so the fault itself is part of what they see.

The tension in this stage

Differencing is the standard remedy for a level shift — but the level shift *is* the fault. Differencing does not hide the fault from the analysis: Stage 5 measures deviation from the baseline separately, and the causal structure is carried in the dynamics, not the level. What differencing does is stop the shared step from making every pair of tags look causally linked simply because they all moved at once.

2.4 Recommended defaults

A reasonable starting point for most industrial data

Linear interpolation for gaps; first-order differencing if any variable fails the stationarity test; standardisation (z-score) for scaling; no smoothing and no resampling unless you have a specific reason. Then check the before/after overlay: the processed trace should still line up with the features of the raw trace. If it does not, you have removed dynamics, not noise.

Symptom	Cause and remedy
Few or no variables stationary	Enable differencing and re-apply. If it still fails, the record may span more than one operating point — shorten it or split the analysis.
Processed trace looks flat	The smoothing window is too large for the dynamics, or scaling was applied to a near-constant tag.
Row count dropped	Expected: differencing and row-dropping consume samples, and resampling divides them. Confirm enough remain for the method you intend.
A message asking you to complete Stage 1	The scenario was never saved.

STAGE 3

Causal Discovery

This is the core of the toolbox: inferring which variables drive which, and with what time delay. Everything after this chapter is read off the graph produced here, so it is worth understanding what the methods are actually testing.

3.1 The single idea behind almost every method

Nineteen algorithms sound like nineteen things to learn. They are not. Almost all of them are variations on one question:

Does knowing the past of A help me predict the future of B , beyond what B 's own past already tells me?

The qualifier is what makes this a causal question rather than a correlational one. Process signals are strongly autocorrelated: a temperature five seconds from now is mostly predictable from the temperature now. Any candidate parent must beat that baseline. If adding A 's history does not improve the forecast of B beyond B 's own history, then whatever the two share is not evidence that A acts on B .

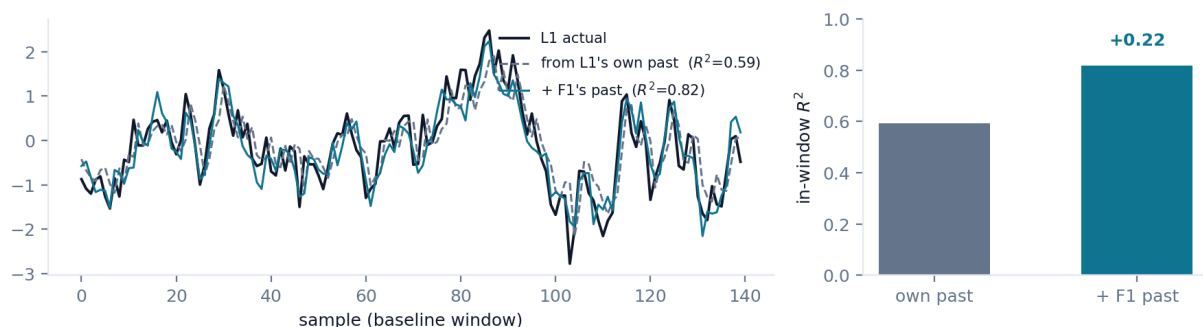


Figure 3.1: The test, performed on the demonstration data. Predicting **L1** from its own past gives $R^2 = 0.59$; adding **F1**'s past raises it to 0.82. That improvement — not the correlation between the two signals — is the evidence for a link **F1** $\rightarrow$ **L1**.

The families differ in *how* they answer the question: with a linear model, a non-linear learner, a neural network, or a repeated resampling procedure. Understanding this makes the whole method list far less intimidating.

3.2 Direct influence versus indirect influence

The second idea worth internalising is *conditioning*, because it is the difference between a graph you can act on and a graph that merely restates the correlation matrix.

In the demonstration process, **T1** $\rightarrow$ **F1** $\rightarrow$ **L1**. **T1** therefore predicts **L1** very well — but only *through* **F1**. A method that examines one pair at a time has no way to know this and reports **T1** $\rightarrow$ **L1** as though it were a physical connection. A method that conditions on every other measured variable asks instead: *does **T1**'s past help predict **L1** once **F1**'s past is already accounted for?* The answer is no, and the indirect link disappears.

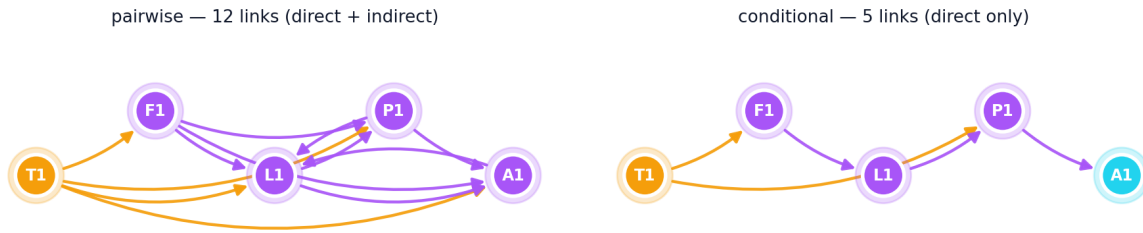


Figure 3.2: The same data analysed two ways. Pairwise testing returns 12 links, including every indirect path and several reversed edges. Full conditioning returns exactly the five true links. Both used the same significance level.

Why this matters for root-cause work

An indirect link is not merely untidy. It shortens the propagation path, so Stage 5 reports the disturbance arriving somewhere it never passed through, and it inflates the out-centrality of upstream variables, distorting the ranking. Prefer conditional methods for anything you intend to act on; use pairwise methods as a fast screen only.

3.3 Domain knowledge

Before running any method, you can tell the tool what you already know. This is the single most effective way to improve accuracy on real plant data, because it prevents the algorithm from rediscovering — and sometimes contradicting — established process physics. Purely data-driven discovery has no concept of piping, flow direction or control logic; you do.

Variable	Upstream order (lower = more upstream; links can't run backward)
T1	1
F1	0
L1	0
P1	0
A1	2

Figure 3.3: The three constraint types. Whatever you enter here is applied to every method, from the classic tests through to deep learning.

3.3.1 The three constraint types

1. Links that cannot exist (forbidden)

Select any ordered pair $A \rightarrow B$ that is physically impossible. The tool removes that link from the result no matter how strong the statistical evidence appears.

When to use it. Whenever a correlation is known to be non-causal: a downstream pressure cannot cause an upstream temperature; an ambient measurement cannot be driven by a vessel level; two sensors on isolated units cannot influence each other. Spurious links of this kind are common when an unmeasured common driver — ambient conditions, feed composition, a shared controller — affects both signals.

2. Links that must exist (required)

Select pairs that are known to be physically coupled. If the method did not find them, the tool adds them with a representative strength.

When to use it. For couplings established by design or by commissioning tests — a control valve and the flow it manipulates, a heater duty and its outlet temperature. Data may fail to reveal such a link when the loop is tightly controlled (the controller suppresses precisely the variation the algorithm needs to see), when the delay exceeds the configured maximum lag, or when the sampling rate is too coarse.

3. Upstream order (tiers)

Assign each variable an integer tier: lower numbers are further upstream. Any link running *backwards* against this order — from a higher tier to a lower one — is removed.

When to use it. This is the most powerful of the three, because a single tier assignment eliminates an entire class of reversed links at once. Derive the tiers from the process flow diagram: feed and utilities first, then each unit operation in sequence, then final product and quality analysers last. Variables may share a tier, which means the tool is free to infer links in either direction between them.

Tier	Example variables	Rationale
0	Feed temperature, feed flow	Enters the unit; nothing downstream can cause it
1	Reactor temperature, pressure	Driven by the feed
2	Separator level, overhead flow	Driven by the reactor
3	Product composition, analyser	Measured last; a pure consequence

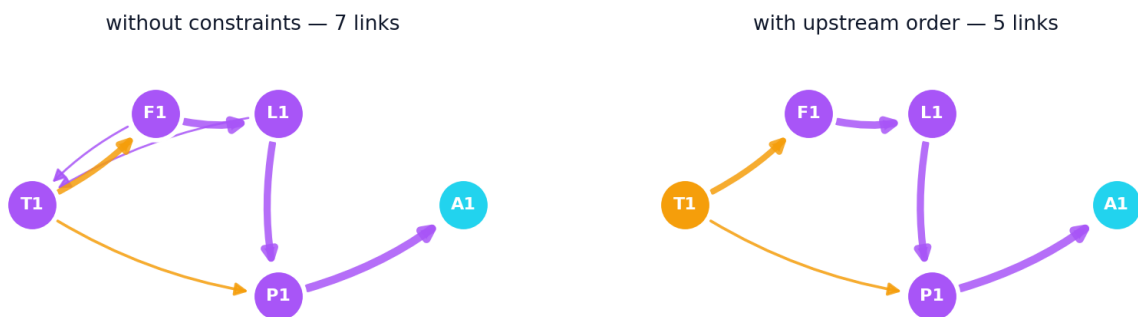


Figure 3.4: Domain knowledge in action, on the demonstration data with the recommended default settings. Left: the unconstrained run returns seven links, two of which ($F1 \rightarrow T1$ and $L1 \rightarrow T1$) run backwards against the process flow — an artefact of differencing, not of the process. Right: a single upstream-order assignment removes exactly those two and leaves the true structure of five links.

3.3.2 How constraints flow through the toolbox

Constraints are not a cosmetic filter on the picture. Once entered they are:

- applied to **every** discovery method, classic through deep learning;
- applied to probabilistic methods as well — a forbidden link has its probability, mean and standard deviation set to zero, and a required link is given probability 1;

- **stored with the graph**, so the reliability tests of Chapter 6 re-run the method under the same constraints and validate what you will actually use;
- carried into the **exported report**, in a “Domain knowledge applied” section, so a reader can see exactly which prior knowledge shaped the result;
- reported in the status line after each run, e.g. “*Domain knowledge applied (3 removed, 1 added)*”. Check those counts against what you entered.

A practical benefit

Constraints also make the structural accuracy metrics usable on real plant data. If you know part of the true topology from a P&ID, you effectively have a partial reference graph — something otherwise available only on the synthetic demonstration dataset.

Use constraints honestly

Constraints are assertions of fact, not preferences. Forbidding a link you merely *dislike*, or requiring one you merely *expect*, will manufacture a graph that confirms your assumptions and hides what the data was trying to tell you.

The consequences are not confined to the picture. A required link that is not real is a false positive in every metric; a forbidden link that is real is a false negative, and it removes a route from the propagation path, so Stage 5 will report the disturbance travelling a way it did not go. Because the reliability checks re-run the method under the same constraints, a confidently wrong assertion will also look perfectly *stable*. Encode only what is genuinely established by process physics.

3.4 Method families

Nineteen algorithms are organised into four families. Selecting a family filters the method list; each method displays its own plain-language description and its own relevant parameters.

3.4.1 Classic / statistical

Well-established statistical tests. Fast, transparent, and backed by decades of literature — but mostly assume *linear* relationships.

Method	Principle, strengths and limitations	
Conditional (VAR)	Granger	Fits a vector autoregression and tests whether <i>A</i> ’s past improves the prediction of <i>B</i> <i>after accounting for every other measured variable</i> . That conditioning is what removes indirect links, as in Figure 3.2. Recommended default — fast, sparse and interpretable. <i>Limitation</i> : linear only.
Pairwise Granger		The same F-test applied to one pair at a time, ignoring all other variables. Very fast. <i>Limitation</i> : cannot distinguish direct from indirect influence, so it returns dense graphs on tightly coupled plants. Useful as a contrast to the conditional version — links that appear in both are more credible.
Lagged cross-correlation		Finds the time shift at which two signals correlate most strongly. Extremely fast and intuitive. <i>Limitation</i> : purely linear, unconditioned, and symmetric in nature — direction is inferred only from the sign of the lag. Use as a first screen on large tag sets, not as evidence.

Method	Principle, strengths and limitations
Transfer entropy	Information-theoretic: how much does knowing A 's past reduce the uncertainty about B 's next value? Model-free and captures <i>non-linear</i> coupling that correlation misses. <i>Limitation</i> : needs a fair amount of data, and results depend on the histogram bin count.
PCMRI	A two-stage constraint-based search: it first prunes candidate parents, then runs conditional independence tests at each lag. Well suited to high-dimensional problems with many variables, where full conditioning loses power. <i>Requires tigramite</i> .

3.4.2 Machine learning — traditional

These replace the linear model inside the Granger test with a non-linear learner, keeping the logic of Section 3.1 intact. For each target variable, a model is trained on the lagged values of all variables. Each source's contribution is then measured by **grouped permutation importance** on a held-out tail: all of that source's lag columns are shuffled together, and the resulting rise in forecast error is recorded. Shuffling breaks any relationship between that source and the target while leaving everything else intact, so a meaningful rise means the source genuinely carried predictive information — and shuffling the lags as a *group* prevents a variable's own lags from substituting for one another and hiding the effect.

Method	Principle, strengths and limitations
Random-Forest Granger	An ensemble of decision trees. Robust, handles interactions and non-linearity with little tuning, and rarely overfits. The best general-purpose non-linear choice. <i>Limitation</i> : slower than the classic tests, and importance values are not comparable across datasets, so the threshold needs calibrating each time.
Gradient-Boosting Granger	Boosted trees, which build sequentially and can capture subtler structure than a forest. Run it alongside random forest as a cross-check. <i>Limitation</i> : more sensitive to noisy data.
SVR Granger (kernel)	A support-vector regressor with an RBF kernel — good for smooth curved relationships and effective on shorter records. <i>Limitation</i> : training cost grows roughly quadratically with sample count, so it is slow on long series.
Lasso-Granger (sparse)	An L1-penalised VAR: the penalty drives weak coefficients exactly to zero, so only the strongest predictors of each variable survive. Produces the cleanest, sparsest graphs. <i>Limitation</i> : linear.
Mutual information (lagged)	Measures how much information A 's past shares with B 's future. Detects arbitrary non-linear association. <i>Limitation</i> : bivariate and unconditioned, so like pairwise Granger it cannot separate direct from indirect links — treat it as a screen.
VarLiNGAM	Exploits <i>non-Gaussian</i> noise to identify causal direction that correlation alone cannot resolve — it can orient a link even where both directions fit the covariance equally well. Powerful when its assumptions hold. <i>Requires lingam</i> ; assumes linearity.

3.4.3 Machine learning — deep learning

Each of these trains a small neural network per target variable, with a **group-lasso** penalty applied to the input weights. The penalty pushes an entire variable's input weights towards zero *together*, so a variable is either retained as a parent or eliminated entirely rather than surviving through one incidental lag. The variables whose weight groups survive are the causal parents. This is the neural-network analogue of Lasso-Granger, and it can represent relationships no linear model can.

Method	Principle, strengths and limitations
Neural Granger — cMLP	A feed-forward network over the lag window. The lightest deep option and a good first choice in this family.
Neural Granger — cLSTM	Long short-term memory units carry state across time, so the model can represent influences acting over <i>long or variable delays</i> that a fixed lag window handles poorly. Slowest of the family.
Neural Granger — cGRU	Gated recurrent units: similar capability to cLSTM with fewer parameters and faster training. A good compromise.
Convolutional Granger	TCN A dilated temporal convolutional network with sparsity on the input channels. Efficient at capturing local temporal patterns over a wide receptive field, and parallelises better than the recurrent options.
Temporal (TCDF) attention	Learns an explicit attention weight per candidate parent, so the model states directly which inputs it relies on. Attractively interpretable.

When deep learning is *not* the right choice

Neural methods need substantial data and compute, and their thresholds require tuning. On a few hundred samples with roughly linear dynamics they will not beat Conditional Granger — they will merely be slower and harder to justify. They also introduce run-to-run variation from random initialisation, so two runs with identical settings may return slightly different graphs. Reach for them when you have long records *and* clear evidence of non-linearity.

3.4.4 Probabilistic

The other three families answer “is there a link?” with yes or no. These answer “*how confident are we that there is a link?*” — every edge weight is a probability between 0 and 1.

Method	Principle, strengths and limitations
Bootstrap edge probability	Re-runs Conditional Granger on many block-resampled copies of the data. Blocks are used rather than individual samples precisely because the time correlation must be preserved — resampling points independently would destroy the dynamics the method is trying to measure. The edge weight is the fraction of resamples in which the link appeared: a direct, assumption-light probability. <i>Limitation:</i> cost is multiplied by the resample count.
Stability (Lasso) selection	Refits the sparse model on many random sub-samples and records how often each link is selected. Excellent at filtering out flukes.

Method	Principle, strengths and limitations
Bayesian regression posterior	Fits a Bayesian linear model and reads the probability that each coefficient group is meaningfully non-zero straight from the posterior. Fastest of the three, since it needs no resampling.

These methods additionally report, for every link, the **mean strength and its standard deviation**. On the causal graph, links with wide uncertainty bands are drawn dashed and dimmed and labelled **mean ± std**; a table beneath the graph lists the same numbers, and both appear in the exported report.

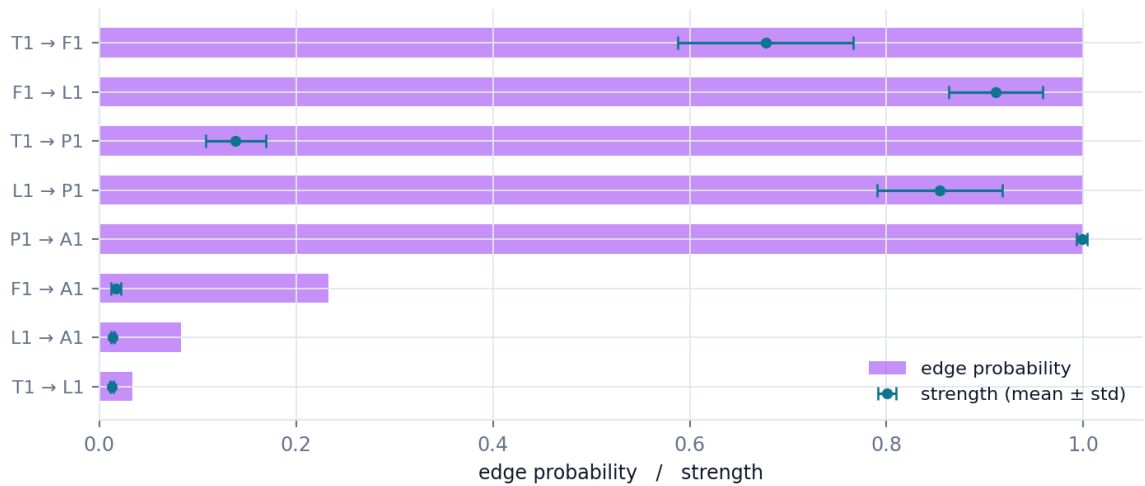


Figure 3.5: Bootstrap output on the demonstration data (60 block resamples). The five true links appear in every resample; the spurious candidates appear in fewer than half, and carry strengths near zero. The separation between the two groups — not the value of any single bar — is what makes this useful.

How to read a probability here

An edge probability of 0.9 does not mean “there is a 90 % chance this link is physically real”. It means the link survived 90 % of resamples *under this method’s assumptions on this data*. A systematically wrong assumption — a linear method on a strongly non-linear process — can produce a confident probability for a link that does not exist. Probability measures stability, not truth.

3.4.5 Choosing between the families

	Classic	ML tradi- tional	ML deep	Probabilistic
Speed	Fast	Moderate	Slow	Moderate–slow
Non-linear	Only TE	Yes	Yes	Partly
Data needed	Modest	Moderate	Large	Moderate
Output	Yes/no	Yes/no	Yes/no	Probability
Interpretability	High	Moderate	Lower	High
Best for	First analysis, routine use	Suspected cur- vature	Long records, complex dynam- ics	Quantified confi- dence

The most reliable practice

No single method is authoritative. Run three or four from *different* families and trust the links they agree on: methods that rest on different assumptions rarely make the same mistake. Agreement between Conditional Granger and Random-Forest Granger is worth far more than agreement between conditional and pairwise Granger, which share nearly all their assumptions. The report's methods-comparison table collects every run for exactly this purpose.

3.5 Parameters

The parameter panel is *method-aware*: it shows only the controls that apply to the selected method, each with a short explanation.

Parameter	Meaning and how to set it
Max lag	How many past time steps a cause may act over. Set it comfortably larger than the slowest expected process delay: a delay beyond the maximum lag is invisible to the method, and the link will simply be missed. Setting it far too large is not free either — each extra lag adds parameters to every model and costs statistical power.
Significance α	Statistical strictness for the classic tests; smaller keeps only stronger, surer links.
Importance / keep threshold	Minimum learned importance for a link to be kept (ML and deep-learning methods). Unlike α this is not a probability and has no universal value — calibrate it by watching how the edge count responds.
Training epochs, hidden units, sparsity λ	Deep-learning training controls; higher λ prunes more links.
Resamples / sub-samples	How many repetitions the probabilistic methods use — more gives steadier probabilities but takes proportionally longer.
Minimum probability	Only links at or above this probability are drawn.

3.6 Reading the causal graph

Node colour encodes structural role:

- **Driver** — has no incoming links; a source of influence. A candidate origin.
- **Relay** — both driven and driving; passes disturbances on.
- **Effect** — only driven; a terminal symptom.
- **Isolated** — no links at all. Either genuinely independent, or its coupling is outside the configured lag range.

Hovering a node reports how many variables it drives and is driven by; hovering an edge reports its strength and lag.

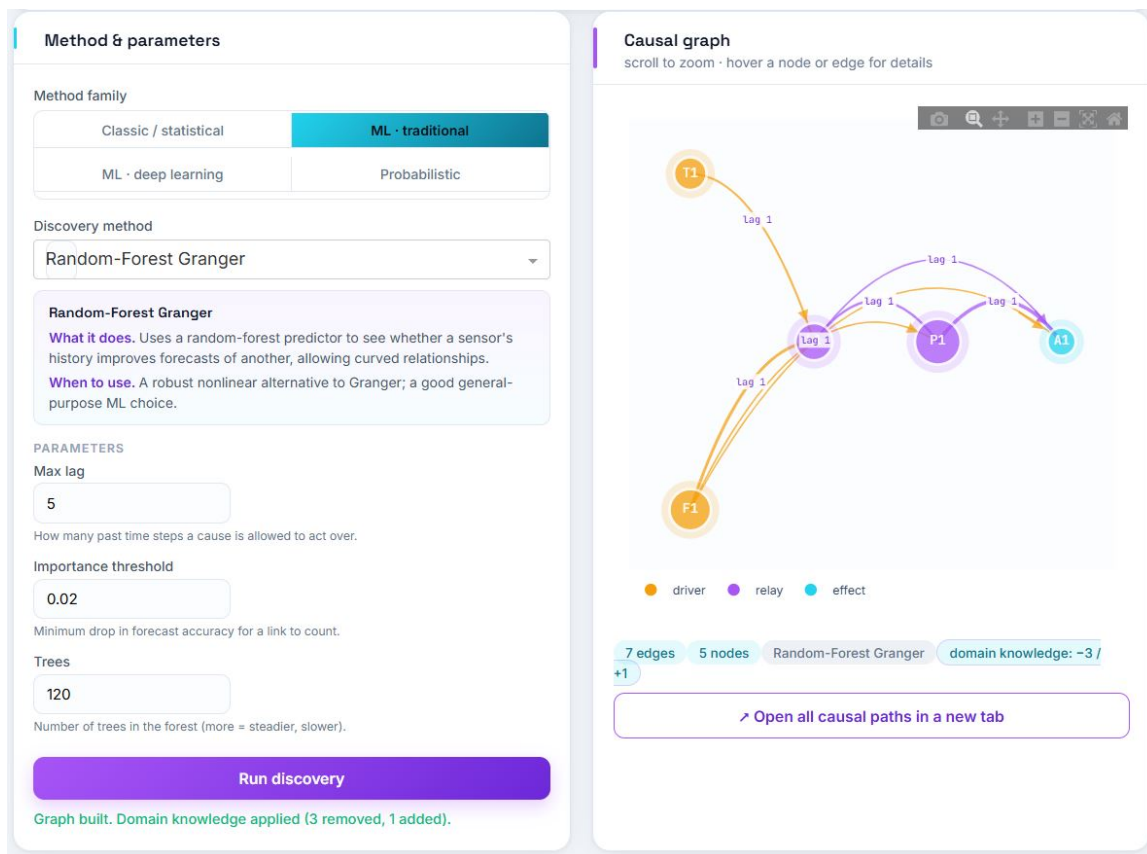


Figure 3.6: A completed run. The left card configures the method; the right card is the result, with summary pills and the full-path export beneath the graph.

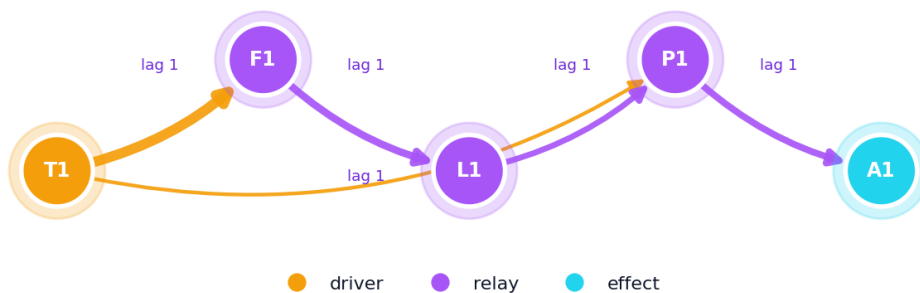


Figure 3.7: The causal graph recovered from the demonstration process by Conditional Granger — exactly the five true links of Figure 2, with no false positives. Arrow thickness encodes strength; the label on each arrow is the estimated delay.

Sanity-check the roles before going further

The roles are the quickest check available on a real plant. Variables you know are upstream should appear as drivers, and analysers should appear as effects. If an analyser is driving the feed, the graph has a direction problem, and the remedy is an upstream order (Section 3.3) rather than a different method.

3.6.1 Full path listing

Because a dense graph can contain dozens of links, **Open all causal paths in a new tab** produces a separate page listing every causal chain in sequence from each initial variable, together with a table of all direct links. That page offers **Download HTML** and **Download CSV**, and is the practical way to read a graph too dense to follow on screen.

3.7 If the graph looks wrong

Symptom	Remedy
Far too dense	Lower α or raise the importance threshold; switch from pairwise to conditional; add domain knowledge.
Empty	Raise α or lower the threshold; raise Max lag ; verify pre-processing did not flatten the signals; check the window is long enough for the number of variables.
Links contradict process flow	Enter an upstream order.
A method is greyed out	Its optional library is missing — tigramite for PCML, lingam for VarLiNGAM, torch for deep learning.
Two methods disagree entirely	Expected to some degree, and informative. Trust the intersection, and check stationarity before trusting either.

STAGE 4

Monitoring

A causal graph computed over one long window is a static picture: it describes the average structure of the whole record. Stage 4 asks a different question — *when* did that structure change? It slides an analysis window across the data, re-runs the entire discovery step inside each position, and compares consecutive results.

Monitoring reuses the method, parameters and domain-knowledge constraints from your last discovery run. It is therefore consistent with the graph you have already inspected, and changing the method in Stage 3 changes what Stage 4 reports.

4.1 What a structural shift is — and is not

This distinction decides whether Stage 4 will tell you anything at all, and it is the most common source of confusion about the page.

- A **level fault** moves a variable without changing what drives what. A valve sticks, a heater trips, a feed composition shifts: the signals move, sometimes dramatically, but *A* still drives *B* with the same delay and the same gain. *The causal graph does not change, and Stage 4 correctly reports nothing.*
- A **structural fault** changes the couplings themselves. A controller is switched to manual and a loop stops responding; a heat exchanger fouls until an upstream temperature no longer reaches a downstream one; a bypass opens and re-routes flow. *Here the graph genuinely changes, and Stage 4 is the instrument that sees it.*

A quiet monitoring result is not a failure

If Stage 4 reports no structural change while you know a fault occurred, the most likely explanation is that the fault was a level fault. That is information, not an error: it tells you the disturbance travelled through the existing causal pathways rather than creating new ones, which is what Stage 5 is designed to trace. Do not raise the sensitivity until the plot shows something; you will only be plotting estimation noise.

4.1.1 Why the change is spread over several windows

In Figure 4.1 the re-routing happens at sample 900, but the change is registered at window centres 850 and 1000, with an elevated link count in between. This is a property of any sliding window, not a defect. A window of 400 samples centred at 900 contains 200 samples of the old structure and 200 of the new; both couplings are partially detectable, so the window reports *six* links where each regime alone has five. The transition therefore appears as a plateau whose width is roughly the window size.

The practical consequences:

- The flagged sample locates the change to within about half a window. Report it as a region, not as a timestamp.
- Narrower windows localise the event more sharply but estimate each graph from less data, which raises the noise floor. This is the central trade-off of the stage.
- An elevated *link count* during the transition, as in the middle panel, is a useful confirmation that a genuine substitution occurred rather than a link simply disappearing.



Figure 4.1: Both cases, run with identical settings (window 400, step 25). **Top two panels:** a variant of the demonstration process in which the $F1 \rightarrow L1$ coupling is replaced by $T1 \rightarrow L1$ at sample 900. The structure genuinely re-routes, and the change is detected — note that it registers over the *region* of windows that straddle the event, not at a single point. **Bottom panel:** the standard demonstration process, whose fault is a step in *level* at the same sample. Structural change is exactly zero throughout, which is the correct answer.

4.2 Settings

Setting	How to choose it
Window size	Samples per analysis window. It must be long enough for the chosen method to estimate a <i>stable</i> graph on its own — if a single window’s graph is unreliable, the difference between two of them is nothing but noise. A few hundred samples is a reasonable start for Conditional Granger; non-linear and neural methods need considerably more.
Step	How far the window advances each time. Smaller steps give finer time resolution at proportionally greater cost. The number of windows is approximately $(N - \text{size})/\text{step} + 1$.

A diagnostic for the settings themselves

Look at the structural-change trace during a period you believe to be normal. It should sit at or near zero. If instead it fluctuates by several links from one window to the next, the window is too short for the method, and no threshold setting will fix that — a peak cannot be distinguished from the floor it sits on. Lengthen the window, or switch to a faster and steadier method such as Conditional Granger, before interpreting anything.

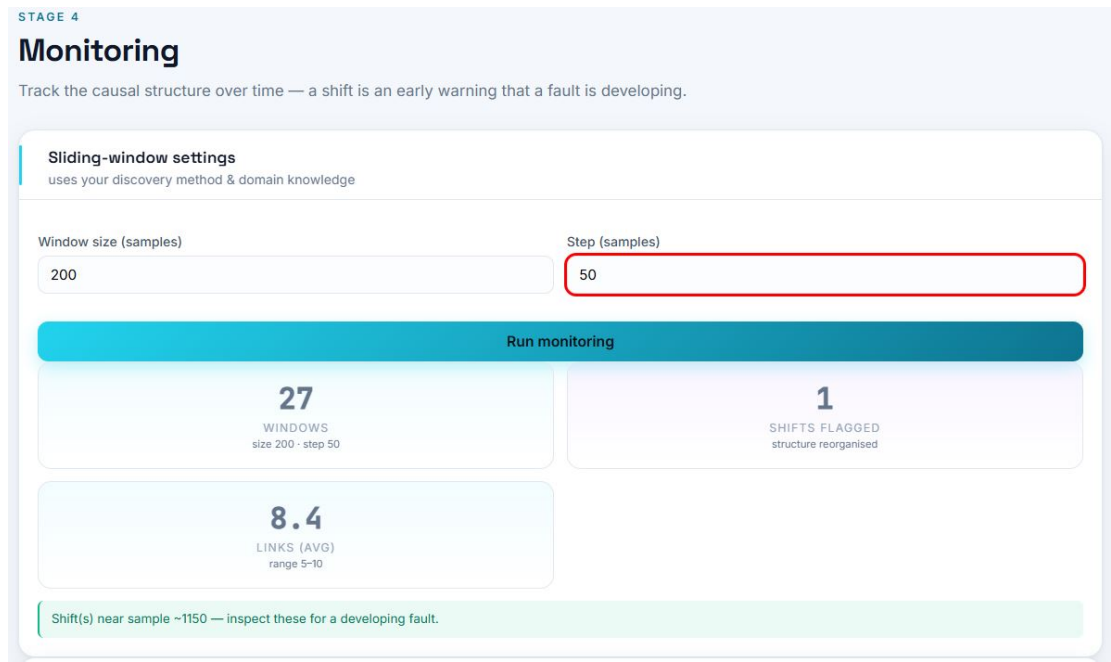


Figure 4.2: The settings and summary card. The tiles report the window count with the settings echoed back, how many windows exceeded the automatic threshold, and the average number of links per window with its range. The run shown uses Random-Forest Granger on the standard demonstration data.

Cost

Monitoring runs a *full discovery per window*. Twenty-seven windows means twenty-seven complete analyses. With heavy methods — deep learning, or any bootstrap-based probabilistic method — and a small step this becomes very expensive, and the tool automatically caps the analysis at 60 windows. For routine scanning, Conditional Granger is the fast and appropriate choice; save the expensive methods for the single detailed run in Stage 3.

4.3 Reading the two plots

The **upper panel** counts how many links differ from the previous window. The **lower panel** plots the raw number of links in each window, and it is worth reading even when the upper panel is quiet: a slow drift in link count means the structure is gradually gaining or shedding couplings, which no window-to-window difference will highlight.

4.4 Link activity

The change count tells you *that* something moved. The link-activity heat map tells you *which* coupling moved, which is the information you can act on.

4.5 If something looks wrong

Symptom	Remedy
Only a handful of windows	The window size or step is large relative to the record. Reduce one or both, keeping the window long enough for the method.

Symptom	Remedy
The run is very slow, or the window count is capped	The 60-window cap has been reached, or the method is heavy. Increase the step, or re-run with Conditional Granger.
Structural change is noisy everywhere	The window is too short for the method to estimate stably. Lengthen it or use a lighter method <i>before</i> interpreting any spike.
No shift flagged although a fault is known	Most likely a level fault, which does not change structure (Section 4.1). Proceed to Stage 5.
The heat map is almost empty	The Stage 3 threshold is too strict for these shorter windows: each window carries far less data than the full-record run did. Loosen it for monitoring, or lengthen the window.



Figure 4.3: The structural-change view in the interface: links changed relative to the previous window (top), the automatic alarm threshold as a dotted line, a star marking a flagged window, and the raw link count per window (bottom).

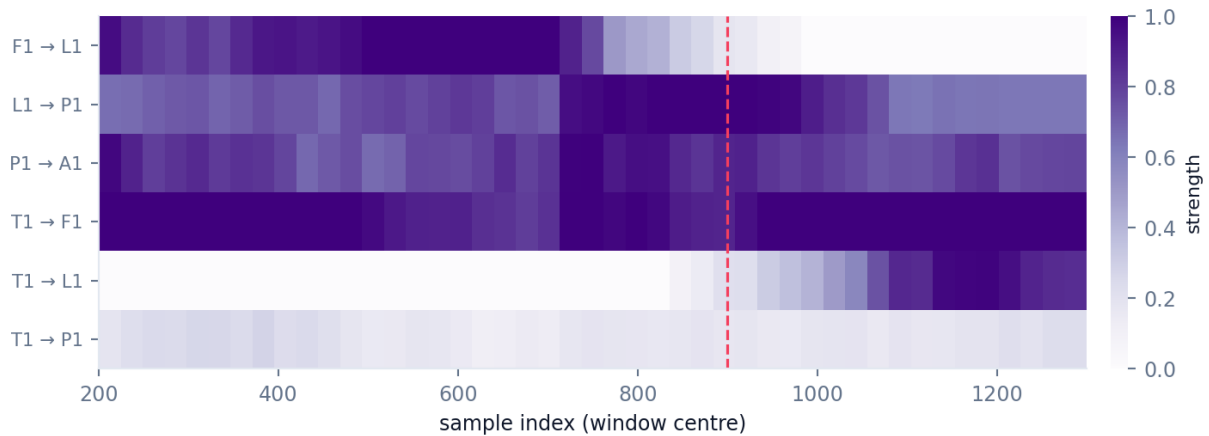


Figure 4.4: Link activity for the re-routing variant: one row per candidate link, one column per window, colour showing the link's strength in that window. The substitution is unmistakable — **F1**→**L1** fades out as the change point is approached, while **T1**→**L1** appears and strengthens afterwards. Rows that stay pale throughout are links the method essentially never found.



Figure 4.5: The same view in the interface. Scan each row horizontally: a row that switches on or off partway through the record is the finding; a row that is uniformly dark or uniformly pale is not.

STAGE 5

Root-Cause Identification

Stage 5 combines two things you already have — the causal graph from Stage 3, and the record of how each variable behaved in the fault window relative to the baseline — and answers the operational question directly: *where did it start, what did it pass through, and how long did each step take?*

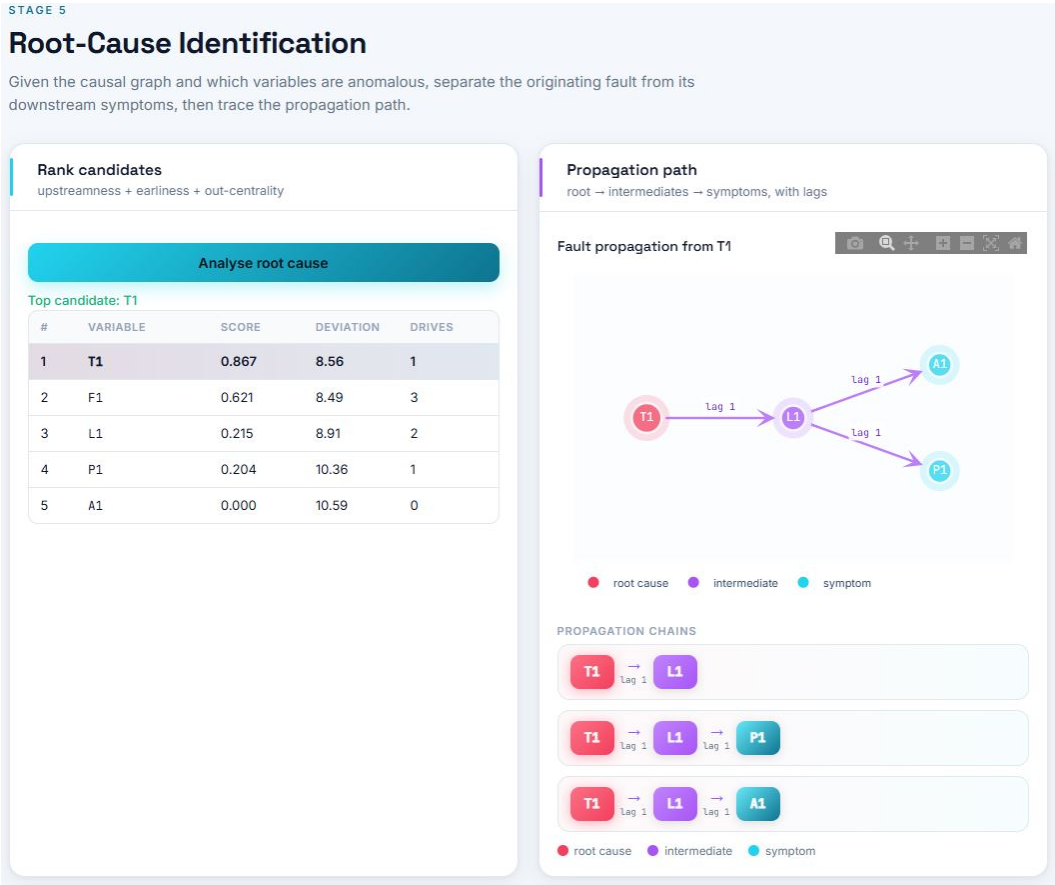


Figure 5.1: Stage 5: ranked candidates on the left, traced propagation on the right. The run shown uses Random-Forest Granger with operator constraints applied, so its graph differs from the worked example in this manual.

5.1 How candidates are ranked

Each variable is scored on three structural and temporal factors, which are combined into a single score between 0 and 1:

Factor	What it measures, and why it is evidence
Earliness	How soon the variable started misbehaving after the fault window opened. A cause cannot begin after its effect, so the earliest mover is a strong candidate and a late mover is nearly disqualified. This is the factor most damaged by setting the fault window too late (Chapter 1).

Factor	What it measures, and why it is evidence
Upstreamness	How far upstream the variable sits in the causal graph. A variable with no parents cannot have been driven by anything measured, so the disturbance either originated there or entered from outside the recorded set.
Out-centrality	How many other variables it drives, directly or through intermediaries — the Drives column in the interface. A genuine origin should be able to explain the variables that moved; something that drives nothing explains nothing.

A fourth quantity, **deviation** — how far the variable departed from its baseline behaviour, in standard deviations — is displayed alongside the score but does not raise it. The next section explains why that design choice is deliberate and important.

5.2 The biggest deviation is usually not the cause

Effects accumulate as a disturbance travels downstream. Each stage of a chain applies its own gain, and each adds its own integrating dynamics, so the variable at the end of the chain typically swings furthest while the variable that actually failed may barely stand out against its own noise. An operator watching a trend display is therefore drawn, reliably and incorrectly, to the last link in the chain.

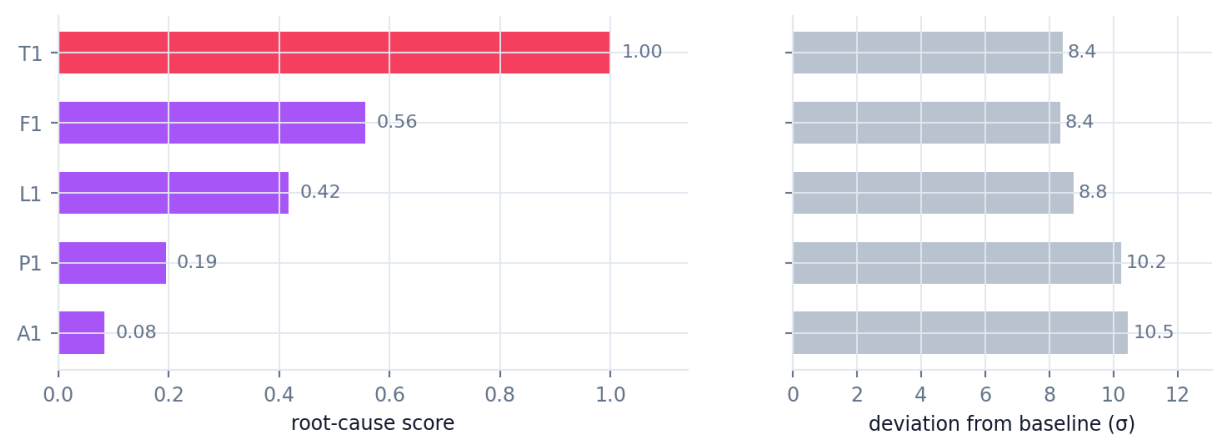


Figure 5.2: The two views side by side for the demonstration fault. **Left:** the root-cause score, which correctly ranks the true origin **T1** first at 1.00. **Right:** the deviation of the same variables from baseline. The ordering is very nearly *reversed*: **T1** deviates by 8.4σ while the terminal analyser **A1** — which scores 0.08 and is a pure symptom — deviates by 10.5σ , the largest excursion in the plant. Ranking on deviation would have named the wrong variable.

What to tell an operator

The sensor screaming loudest is usually the one furthest from the problem. That is not a quirk of this tool; it is a property of how disturbances travel through coupled processes, and it is the reason a causal method is needed at all rather than a ranked list of alarm magnitudes.

5.3 Propagation path

Having identified the most likely origin, the tool traces every route from it through the graph, annotating each step with its estimated delay.

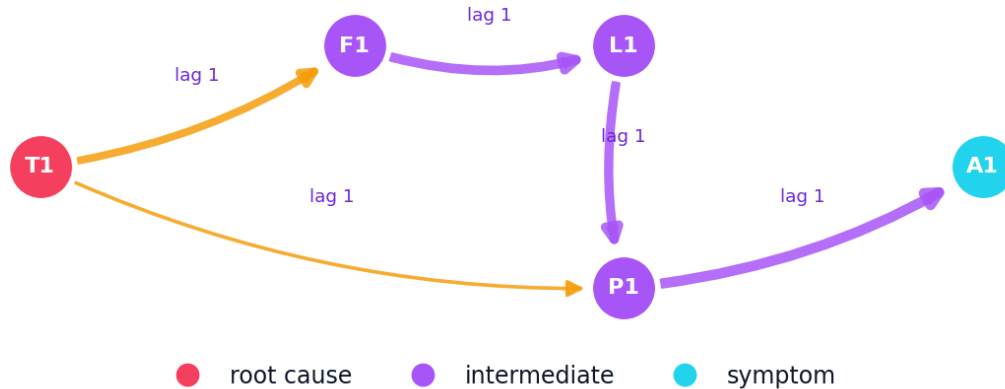


Figure 5.3: Traced propagation for the demonstration fault: the root cause (red) on the left, intermediates (violet) in the middle, terminal symptoms (cyan) on the right, with the estimated time lag on each step.

For the worked example the tool returns two chains from T1:

Chain	Interpretation
T1 → P1 → A1	the shortcut route
T1 → F1 → L1 → P1 → A1	the main chain

Both are correct: the demonstration process contains exactly these two paths, and recovering both — rather than only the longer one — is what allows the delays to be interpreted. When two routes of different length converge on the same variable, that variable will show a smeared or two-stage response, which is often visible on the trend and is a useful confirmation that the graph is right.

5.3.1 Reading the chains operationally

A propagation chain is a hypothesis about where to look, in order. The first link is where to inspect equipment; the intermediates are where to expect confirming evidence; the terminal symptoms are what you can safely ignore when deciding what to fix. If the chain passes through a variable you know to be uninvolved, that is a signal to revisit the graph — most often by adding a constraint (Section 3.3) rather than by dismissing the result.

The path can only be as good as the graph

Every constraint you entered in Stage 3 propagates into this view. Forbidding a link removes every chain that would have passed through it; requiring one creates chains that the data alone did not support. If a variable you expect to be involved is missing from the propagation entirely, check your constraints before questioning the process.

5.4 Plain-language summary

Graphs and score tables require interpretation. For handover notes, shift logs and management reporting, a paragraph of ordinary prose is more useful — so the tool writes its own narrative summary of the analysis.

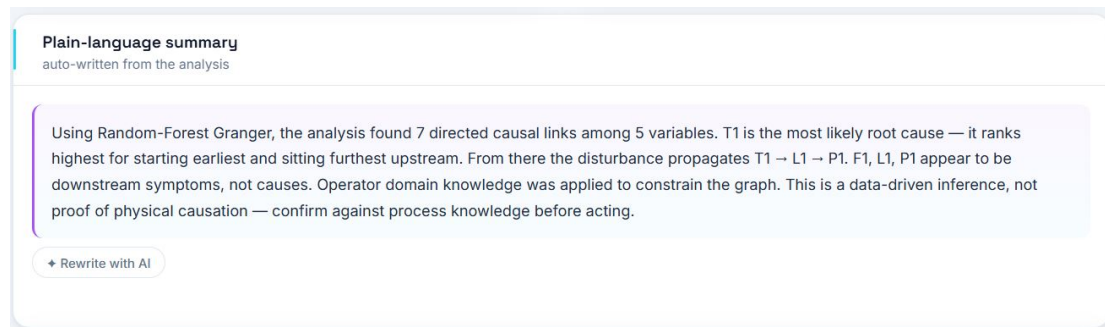


Figure 5.4: The summary card. It refreshes automatically whenever a stage is re-run, appears on the Root-Cause page, and is placed at the top of both the HTML and PDF reports.

5.4.1 What it contains

The summary is assembled from whatever you have actually computed, in this order:

1. the method used and how many links it found;
2. the identified root cause and *why* it ranked first;
3. the propagation chain from the root;
4. which variables are downstream symptoms rather than causes;
5. the reliability evidence, with a plain verdict on how far the graph can be trusted — included only if you have run those checks;
6. a note if domain-knowledge constraints were applied;
7. a standing caution that this is inference, not proof.

*Using Conditional Granger (VAR), the analysis found 5 directed causal links among 5 variables. **T1** is the most likely root cause — it ranks highest for starting earliest and sitting furthest upstream. From there the disturbance propagates **T1** → **F1** → **L1** → **P1** → **A1**. **F1**, **L1**, **P1**, **A1** appear to be downstream symptoms, not causes. On reliability, the discovered parents explain about 42% of out-of-sample variation and add +0.40 on average over each variable's own history; every discovered link survived resampling; a random dummy variable stayed unconnected — so the structure can be treated with confidence. Operator domain knowledge was applied to constrain the graph. This is a data-driven inference, not proof of physical causation — confirm against process knowledge before acting.*

5.4.2 Why it is generated deterministically

The summary is template-driven, assembled from the stored numerical results — it is *not* produced by a language model. For an industrial tool this matters in three concrete ways:

- it requires **no API key, no internet and no external service**, so it works on an isolated plant network;
- it **cannot hallucinate** — every number in it is read directly from the computed results, and if a check has not been run the corresponding sentence is omitted rather than guessed;
- it is **reproducible** — the same analysis always yields the same wording, which is a prerequisite for an auditable record.

5.4.3 Optional AI rewrite

The **Rewrite with AI** button passes the generated summary to whichever language model you have configured (Chapter 7) and asks for a more fluent operator briefing of three to five sentences. The instruction explicitly constrains the model to the supplied facts.

Review AI-rewritten text before circulating it

The rewrite is a *presentation* aid, and it is the one place in the pipeline where text is generated rather than computed. A language model can subtly shift emphasis or overstate confidence — “the structure can be treated with confidence” becomes “the analysis confirms” — so read the result against the deterministic version, which remains available and unchanged, before putting it into a formal record. If no model is configured, the button reports that a key is needed and the deterministic summary is untouched.

5.5 Understanding the score in the interface

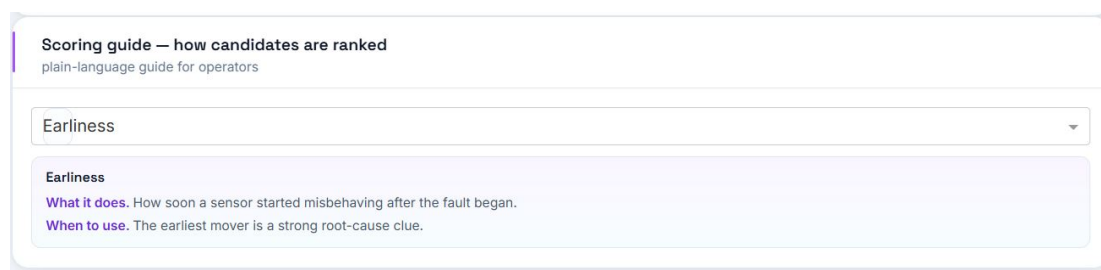


Figure 5.5: Every scoring factor explains itself in plain language. This card is designed for handing the screen to someone who was not part of the analysis.

STAGE 6

Evaluation & Report

6.1 The central difficulty

On a real plant the true causal graph is unknown. That is not a minor inconvenience to be noted and moved past — it is the defining problem of applied causal discovery. Every metric that compares a discovered graph against a true one is unavailable exactly when you most need it, and any tool that reports precision and recall on plant data is either using a reference you supplied or is reporting something meaningless.

The toolbox therefore separates two questions that are easily confused:

- **Is this graph trustworthy?** Answerable without a reference, using the four reliability checks below. This is your primary evidence on real data.
- **Is this graph correct?** Answerable only against a known structure — the synthetic demonstration, or a plant whose topology you know from a P&ID.

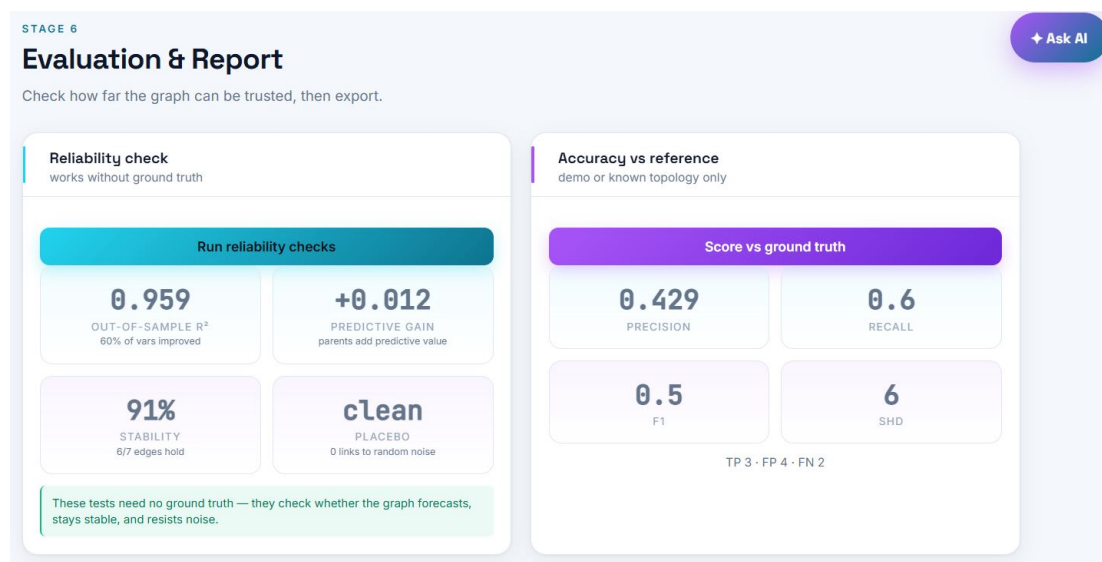


Figure 6.1: Stage 6. The left card works on any dataset; the right card requires a reference structure and stays empty without one. The run shown is a Random-Forest Granger analysis with operator constraints, not the worked example of this manual.

6.2 Reliability — validation without ground truth

6.2.1 Out-of-sample R^2

What it measures. How well each variable is predicted on data the model never saw, using its discovered parents.

How to read it. A value near 1 means the parents explain the variable almost completely; near 0 means they explain little. Judge it *relative to the process*: a noisy analyser will never reach the R^2 of a smooth temperature, and a low value on an intrinsically noisy tag is not evidence against the graph.

What it cannot tell you. On its own, almost nothing. Process signals are strongly auto-correlated, so a high R^2 is often achievable from a variable's own past alone, with the discovered parents contributing nothing. That is why the next number exists.

6.2.2 Predictive gain

What it measures. The improvement over predicting each variable from its *own past alone*. **This is the single most important reliability number on the page, and the one to read first.**

Why it matters. A causal parent must carry information about its child that the child's own history does not already contain. If adding the discovered parents does not improve out-of-sample forecasts, the links are decorative — they may be stable, they may be numerous, and they are still not doing any work.

How to read it. Positive across most variables is good. Near zero overall means the graph is not capturing genuine structure: change method, re-check stationarity, or lengthen the record. A high R^2 combined with a near-zero gain is the classic trap, and it means the autocorrelation is doing all the work.

A zero gain can be the correct answer

The *root cause* should show no gain, because it has no parents to improve its forecast. In Figure 6.2, T1 gains exactly +0.00 while every downstream variable gains between +0.37 and +0.59. That pattern — one variable flat, all others lifted — is the signature of a correctly identified origin, and it corroborates Stage 5 independently.

6.2.3 Bootstrap stability

What it measures. The data is resampled many times in *contiguous blocks* — blocks rather than individual samples, so that the time correlation that the whole analysis depends on is preserved — the same method is re-run on each resample, and the tool records how often each discovered link reappears.

How to read it. Above roughly 70% is a solid link. Links appearing in only a minority of resamples are probably artefacts of the particular sample and should not drive a decision. The summary also reports how many individual edges cleared the threshold, e.g. *5/5 edges hold*.

What it cannot tell you. Whether the links are *right*. Stability measures consistency, and a method can be consistently wrong — particularly when a constraint has forced a link, or when a systematic pre-processing artefact is present in every resample.

6.2.4 Placebo test

What it measures. A column of pure random noise is injected into the data and discovery is re-run. A trustworthy method connects it to nothing.

How to read it. *Clean* is the expected result. Any link to the placebo is a *provable* false positive — there is no mechanism by which random noise could drive a process variable — which means the current settings are too permissive. Lower α or raise the importance threshold and re-run.

6.2.5 Reading the four together

No single tile is a verdict. The combinations are:

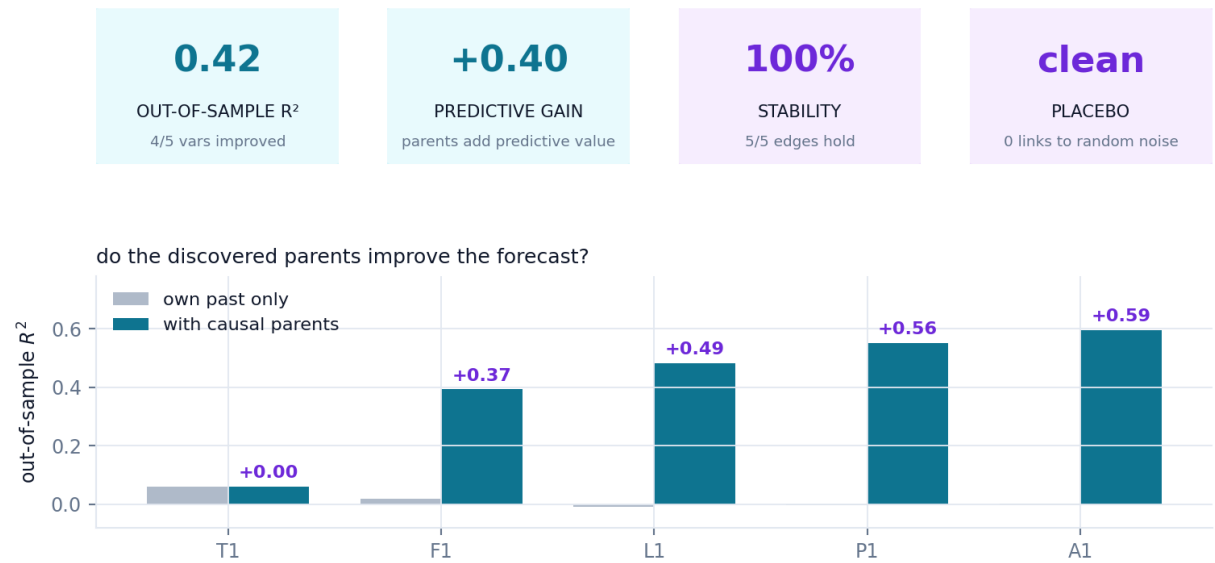


Figure 6.2: Reliability results for the worked example. **Top:** the four summary tiles. **Bottom:** out-of-sample R^2 from each variable’s own past alone (grey) against the same forecast with its discovered causal parents (teal), with the gain printed above each pair. Every downstream variable improves substantially; the root **T1**, which correctly has no parents, gains nothing.

Pattern	Interpretation and action
High gain, high stability, clean placebo	The graph is trustworthy. Proceed to root-cause analysis with confidence. This is the worked example’s result.
High gain, low stability	The structure is real but the specific links are uncertain — often the case on short records. Use a probabilistic method to quantify which links are dependable, and report the uncertainty rather than a single graph.
Low gain, high stability	Consistently finding the same links, which carry little predictive information. Suspect a linear method on non-linear data, an over-smoothed record, or constraints doing the work the data should be doing.
Placebo not clean	Settings are too permissive. Tighten the threshold and re-run discovery before interpreting anything else on the page.
High R^2 , near-zero gain	The autocorrelation is doing all the work and the links are decorative. Treat the graph as unproven regardless of how plausible it looks.

6.3 Structural accuracy — reference required

When a reference graph *is* available, four classical metrics apply. All compare the discovered link set against the true one.

Metric	Definition	How to read it
Precision	Of the links reported, the fraction that are correct.	Low precision means the graph invents links. Tighten α or raise the threshold.

Metric	Definition	How to read it
Recall	Of the links that truly exist, the fraction that were found.	Low recall means real connections were missed. Loosen α , raise the maximum lag, or try a non-linear method.
F1	The harmonic mean of the two (1.0 is perfect).	The single “how good is this graph” number. Use it to rank methods against one another on the same data.
SHD	Structural Hamming Distance: the number of edits — add, remove or reverse — needed to turn the discovered graph into the true one.	Lower is better; 0 is an exact match. Unlike F1 it counts a <i>reversed</i> arrow as one error rather than two, which matters greatly for root-cause work, where a reversal is the most damaging possible mistake.

6.3.1 A worked comparison

Because the demonstration dataset has a known structure, it can show precisely what domain knowledge buys. Both runs below used identical data, identical pre-processing and identical settings; they differ only in whether an upstream order was supplied.

Run	Links	Precision	Recall	F1	SHD
Without constraints	7	0.71	1.00	0.83	2
With upstream order	5	1.00	1.00	1.00	0

Both runs found every true link — recall is perfect either way. The difference is entirely in the two spurious backward links that differencing induces (Section 3.3), and a single tier assignment removes both. Note also what *no* setting of α achieves here: tightening the threshold enough to drop those two links would eventually start dropping true ones as well, because their statistical evidence is not the weakest in the set. Prior knowledge solves in one step what parameter tuning cannot solve at all.

Precision and recall pull against each other

Tightening a threshold raises precision and lowers recall; loosening it does the reverse. There is no universally correct setting — but for root-cause analysis a slightly denser graph is usually safer, because a *missing* link can break the propagation chain entirely and hide the origin, whereas an *extra* link is visible on the graph and can be dismissed by an operator who knows the plant.

Do not report these on real plant data

Without a known reference graph these four numbers cannot be computed, and any value claiming otherwise is meaningless. Use the reliability checks instead. If you know part of the topology from a P&ID you have a *partial* reference, which is genuinely useful — but say so explicitly whenever you quote the resulting figures, because they describe only the part you supplied.

6.4 The report

The report card summarises the run and lists **every discovery method you have used**, with edge counts and — where a reference exists — F1 and SHD, so that the export reflects your

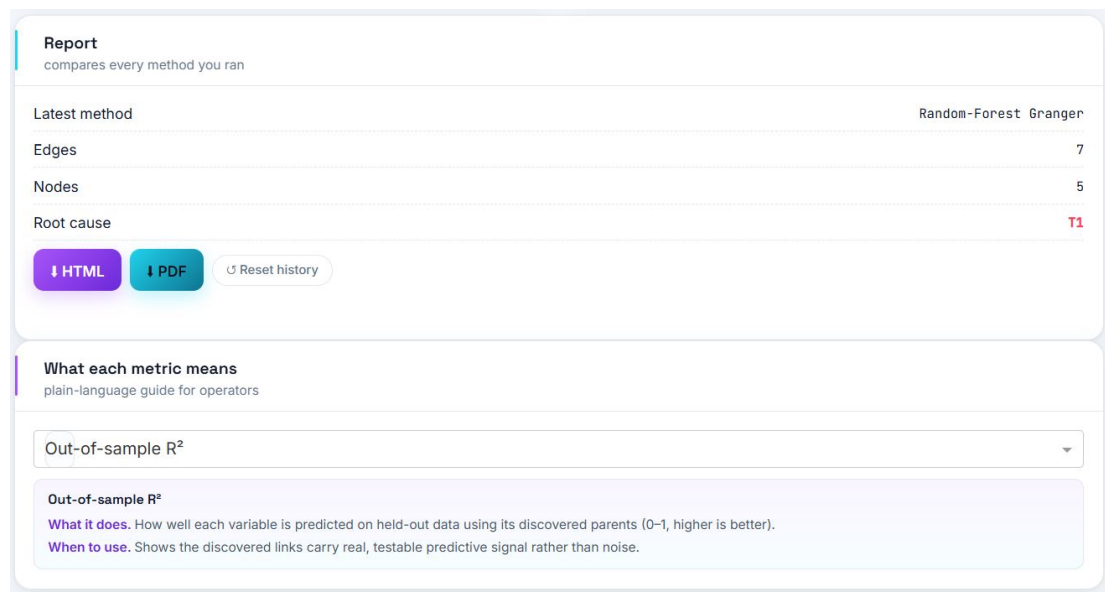


Figure 6.3: The export card and the metric glossary that accompanies it. The summary rows describe the latest run; the report itself covers every method used in the session.

whole investigation rather than only the last run. This is what makes the multi-method practice recommended in Chapter 3 auditable: a reader can see which methods agreed.

Two formats are available:

- **HTML** — self-contained, with an interactive causal graph. Suitable for circulation by e-mail and for embedding in a wiki.
- **PDF** — print-ready, with a natively drawn graph. Suitable for a formal incident record.

Both contain: the narrative summary, the configuration, the causal graph, the causal paths in sequence, applied domain knowledge, per-edge uncertainty for probabilistic methods, the methods comparison, every metric used *with its plain-language description*, the root-cause ranking, and the traced propagation.

Reset history clears the accumulated record of methods run. Use it when beginning a genuinely new investigation — not between methods within one investigation, since comparing them is the point.

Complete the run before exporting

The metrics section is populated from what you have actually computed. Run the reliability checks — and, on the demo, the structural evaluation — before exporting, or the report will have gaps precisely where its strongest evidence belongs.

CHAPTER 7

The AI Assistant

Every page of the toolbox carries a floating **Ask AI** button in the top-right corner. It opens a chat panel connected to a large language model that has been given a summary of your current analysis, so you can ask questions about *your* results in ordinary language.

Entirely optional

The assistant is a convenience layer, not a dependency. Every analytical capability — including the plain-language summary of Section 5.4 — works fully without it. If you never configure a provider, nothing else in the toolbox is affected.

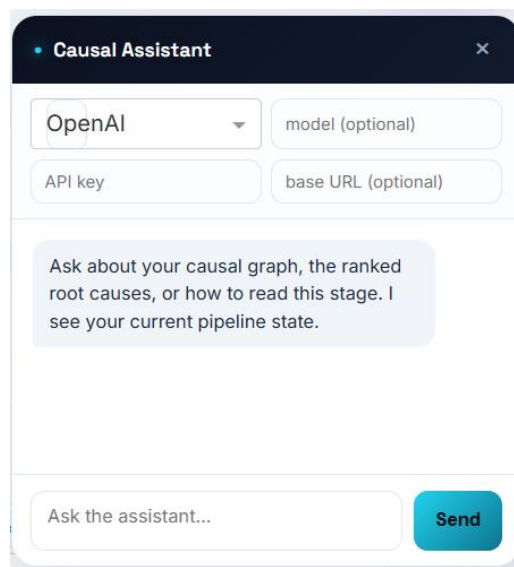


Figure 7.1: The assistant panel before a provider has been configured. The base URL field is for an in-house gateway or a local Ollama server.

7.1 Configuration

Provider	Requires	Notes
OpenAI	API key, model name	Cloud service; data leaves your network.
Anthropic	API key, model name	Cloud service; data leaves your network.
Ollama	A local Ollama server	Runs entirely on your own machine — no data leaves the site. The right choice for confidential plant data.
Custom endpoint	Base URL, key, model	For an in-house or corporate gateway exposing an OpenAI-compatible API.

Data confidentiality

With a cloud provider, the context described below — variable *names*, the method used, edge counts and the root-cause ranking — is transmitted to that provider. Raw process data is never uploaded, but tag names alone may be sensitive: they can reveal unit configuration, capacity and product. If your site restricts external data transfer, use **Ollama** to keep everything local, or leave the assistant unconfigured.

7.2 What the assistant knows

Before each question, the toolbox builds a compact context describing the current state of your analysis:

- the selected variables, baseline window and fault window;
- which pre-processing steps were applied;
- the discovery method used and how many links it found;
- whether domain-knowledge constraints were applied, and of what kind;
- the top root-cause candidates and their scores.

Because of this the assistant answers about *your* run rather than in generalities, and the context is refreshed automatically as you advance through the stages.

7.3 When to use it

Situation	Example question
Understanding a result	<i>“Why is T1 ranked above P1 when P1 deviates more?”</i>
Choosing a method	<i>“My data looks non-linear and I have 5000 samples — which method should I try next?”</i>
Interpreting metrics	<i>“My stability is 45%. Is that acceptable, and what should I change?”</i>
Deciding on constraints	<i>“The graph says the analyser drives the feed temperature. Should I forbid that link?”</i>
Troubleshooting	<i>“My graph came out empty — what usually causes that?”</i>
Drafting a write-up	<i>“Turn these findings into three sentences for a shift handover.”</i>

7.4 When *not* to rely on it

Limitations

- The assistant sees a *summary* of the analysis, not the raw data. It cannot recompute anything or verify a number for you.
- Language models can state incorrect things fluently. Treat its answers as suggestions from a knowledgeable colleague — helpful, not authoritative.
- It has no knowledge of your specific plant beyond what the context contains. It cannot know your equipment, your control strategy or your operating limits.
- **Never let it substitute for the numbers.** The reliability checks of Chapter 6 are

evidence; the assistant’s opinion is not.

Ask it to *explain* rather than to *decide*: “what does this metric mean for my case” works far better than “what should I do”. Ask one focused question at a time; the context is compact, and short questions get sharper answers. If an answer seems doubtful, check it against the corresponding section of this manual or the inline description in the interface, both of which are authoritative.

CHAPTER 8

Running and Deploying the Toolbox

8.1 Development mode versus production

Started directly with `python app.py`, the application runs Dash’s development server with debugging enabled. That is the right configuration while you are developing or evaluating the tool, and the wrong one for anything an operator uses. Debug mode reloads the application whenever a file changes, exposes internal detail in error pages, and adds the floating developer panel described below.

```
# development
python app.py # debug server on port 8051

# production (Linux)
gunicorn --bind 0.0.0.0:8051 --workers 2 --timeout 300 app:server

# production (Windows)
waitress-serve --port=8051 app:server
```

A generous timeout matters: a single deep-learning discovery run or a bootstrap-based reliability check can occupy a worker for minutes, and a default 30-second timeout will kill it mid-analysis.

8.2 The developer panel

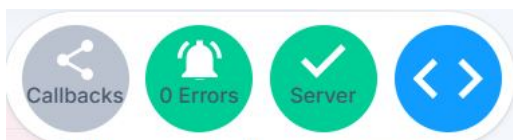


Figure 8.1: The Dash developer panel, expanded. Collapsed, it appears as a single blue `<>` circle in the bottom-right corner.

This cluster belongs to Dash, the framework the toolbox is built on, and appears only in debug mode. It displays nothing about your analysis and cannot change a result.

- `<>` — opens and closes the panel.
- **Callbacks** — a dependency graph of the application’s internal callbacks; of interest when developing the tool, not when using it.
- **Errors** — how many internal errors have occurred in this browser session. Click to read the traceback.
- **Server** — the development server’s status.

The one time it is useful to a user

If the interface misbehaves — a button does nothing, a plot fails to appear — open the panel and copy the traceback before restarting. Sent together with the steps that produced it, that turns “it broke” into something that can be fixed.

8.3 What to expect in terms of run time

Times below are indicative, for five variables and 1 500 samples on an ordinary laptop. They scale roughly with the number of variables squared and linearly with record length.

Operation	Order of magnitude	Scales badly with
Classic discovery (one run)	well under a second	number of variables
Traditional-ML discovery	seconds	record length, variables
Deep-learning discovery	minutes	epochs, record length
Probabilistic (bootstrap)	resample count $\times$ one run	resample count
Monitoring	window count $\times$ one run	small step, heavy method
Reliability checks	comparable to one bootstrap run	resample count

8.4 Data handling

All analysis is performed in the application process; no process data is transmitted anywhere unless you explicitly configure a cloud AI provider (Chapter 7). Uploaded files are held in the session and are not written to a database. Run history for the report's method comparison is kept in the browser's local storage, which is why **Reset history** is a per-browser action.

CHAPTER 9

Reference

9.1 Choosing a method

Situation	Suggested method
First analysis / general use	Conditional Granger (VAR)
Suspected non-linear couplings	Random-Forest Granger, transfer entropy
Many variables, want a sparse graph	Lasso-Granger
Need a confidence per link	Bootstrap edge probability
Long or complex time delays	Neural Granger cLSTM or cGRU
Quick screen of a large tag set	Lagged cross-correlation
Non-Gaussian residuals, linear process	VarLiNGAM
Many variables, constraint-based approach	PCMCI

9.2 Recommended workflow

1. Load data; set baseline and fault windows against the preview.
2. Pre-process: interpolate gaps, difference if trending, standardise. Confirm the stationarity table before continuing.
3. Enter any domain knowledge you genuinely have (Section 3.3).
4. Run Conditional Granger first; inspect the graph and its node roles.
5. Run two or three further methods from *different* families; links that several methods agree on are the most credible.
6. Run a probabilistic method to obtain per-link confidence.
7. Check the causal structure over time in Stage 4.
8. Identify the root cause and read the propagation path.
9. Run the reliability checks — predictive gain first.
10. Export the report.

9.3 Troubleshooting

Symptom	Remedy
Graph is far too dense	Lower α , raise the importance threshold, use conditional rather than pairwise methods, or add domain-knowledge constraints.
Graph is empty	Raise α , lower the threshold, verify pre-processing did not flatten the signals, and check the window is long enough.

Symptom	Remedy
Every variable non-stationary	Enable first-order differencing in Step 4 and re-apply. If it persists, the record may span more than one operating regime.
A method shows “install to enable”	Its optional library is missing: <code>tigramite</code> for PCMCI, <code>lingam</code> for VarLiNGAM, <code>torch</code> for the deep-learning family.
Deep-learning methods are very slow	Reduce epochs, shorten the window, or use a traditional-ML method.
Near-zero predictive gain	The discovered links may be spurious; try another method, re-check stationarity, or extend the record before acting on any root cause.
Structural accuracy unavailable	Expected on real data — no reference graph exists. Use the reliability checks.
Monitoring shows nothing	Likely a level fault rather than a structural one (Section 4.1).
Root cause is a known symptom	The graph has a direction wrong. Enter an upstream order and re-run.

9.4 Interpretation and limitations

Read before acting on results

Causal discovery from observational data yields *plausible* causal structure, not proof. Its conclusions are conditional on the recorded variables, the chosen pre-processing, and the assumptions of the selected algorithm. In particular:

- **Unmeasured common drivers.** If an unrecorded variable — ambient conditions, feed composition, a shared controller — drives two measured signals, they will appear causally linked to each other. No method operating on the recorded set can rule this out; only adding the missing measurement, or knowing the physics, can.
- **Delays beyond the configured lag.** If the true delay exceeds **Max lag**, the link is missed entirely and silently. There is no warning, because the tool cannot see what it did not look for.
- **Controlled loops.** Strong feedback control suppresses exactly the variation an algorithm needs in order to see a coupling, and can mask causal direction because the controller keeps variables in step. A well-controlled loop may show *no* link where the strongest physical coupling in the plant exists.
- **Sampling.** Couplings faster than the sampling interval appear instantaneous, and direction cannot be recovered from simultaneity.
- **Regime changes.** A record spanning more than one operating point yields an average of structures that never coexisted.

Always corroborate against process knowledge, and where the decision matters, against a controlled test. A single deliberate step test settles what months of observational data cannot.

9.5 Glossary

Term	Meaning in this tool
Baseline window	The stretch of record declared to be normal operation; every deviation figure is measured against it.
Conditioning	Testing a link while accounting for all other measured variables. What separates direct from indirect influence.
Driver / relay / effect	Node roles: no incoming links, both incoming and outgoing, only incoming.
Edge probability	For probabilistic methods, the fraction of resamples in which a link appeared.
Lag	The delay, in samples, at which a cause acts on its effect.
Level fault	A disturbance that moves signals without changing the causal structure.
Out-centrality	How many variables a node drives, directly or through intermediaries.
Placebo	A column of random noise injected to test whether the method invents links.
Predictive gain	The out-of-sample forecast improvement contributed by a variable's discovered parents, over its own past alone.
SHD	Structural Hamming Distance: edits needed to turn the discovered graph into the reference.
Stationarity	The property that a series' statistical behaviour does not drift over time; assumed by nearly every method here.
Structural fault	A disturbance that changes the couplings themselves; the thing Stage 4 detects.
Tier / upstream order	An integer per variable expressing how far upstream it sits; links running backwards against it are removed.

9.6 Software requirements

Component	Purpose
<code>dash[diskcache]</code> , <code>plotly</code>	Application framework and plots
<code>pandas</code> , <code>numpy</code> , <code>scipy</code>	Data handling and numerics
<code>scikit-learn</code>	Traditional machine-learning methods
<code>statsmodels</code>	Stationarity tests, pairwise Granger
<code>networkx</code>	Graph algorithms
<code>dash-cytoscape</code>	Interactive causal graph
<code>reportlab</code>	PDF export
<code>torch</code>	Deep-learning methods (a CPU wheel is sufficient)
<code>gunicorn</code> or <code>waitress</code>	Production serving
<code>tigramite</code> , <code>lingam</code>	Optional: PCMCI, VarLiNGAM

Causal RCA Toolbox — companion to the FDI Analytics Toolbox